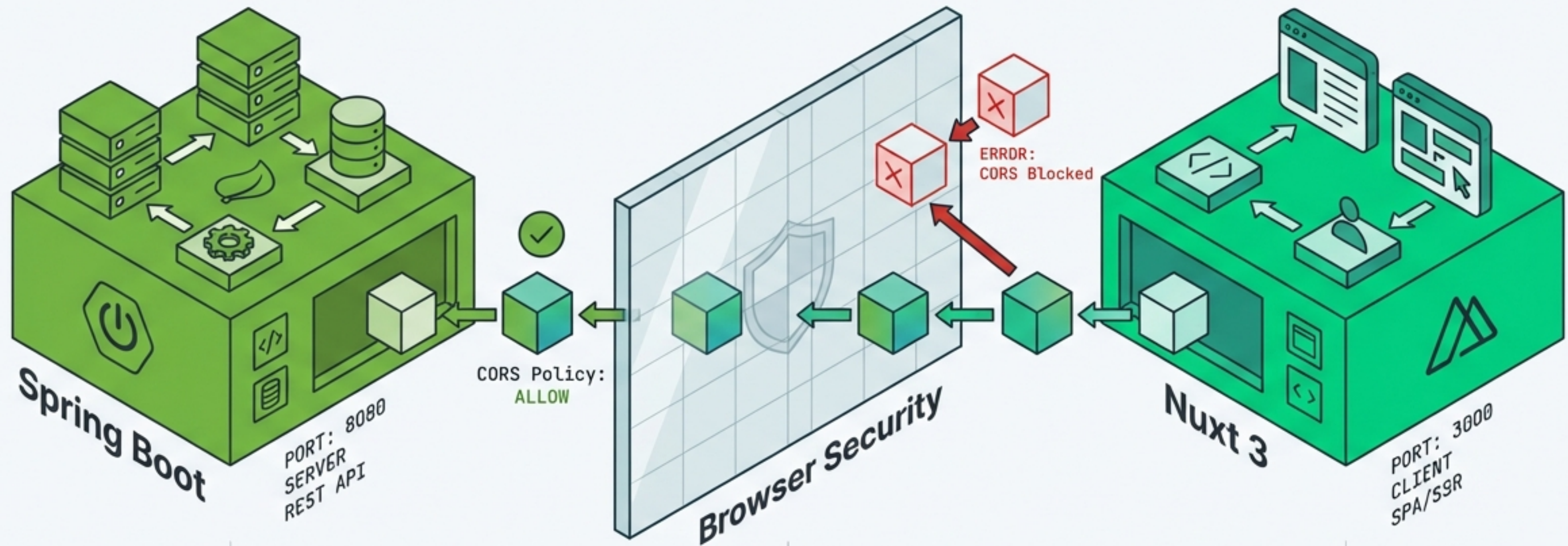


Mastering the Full-Stack Handshake

A Definitive Configuration Guide for CORS in Spring Boot 3 & Nuxt 3



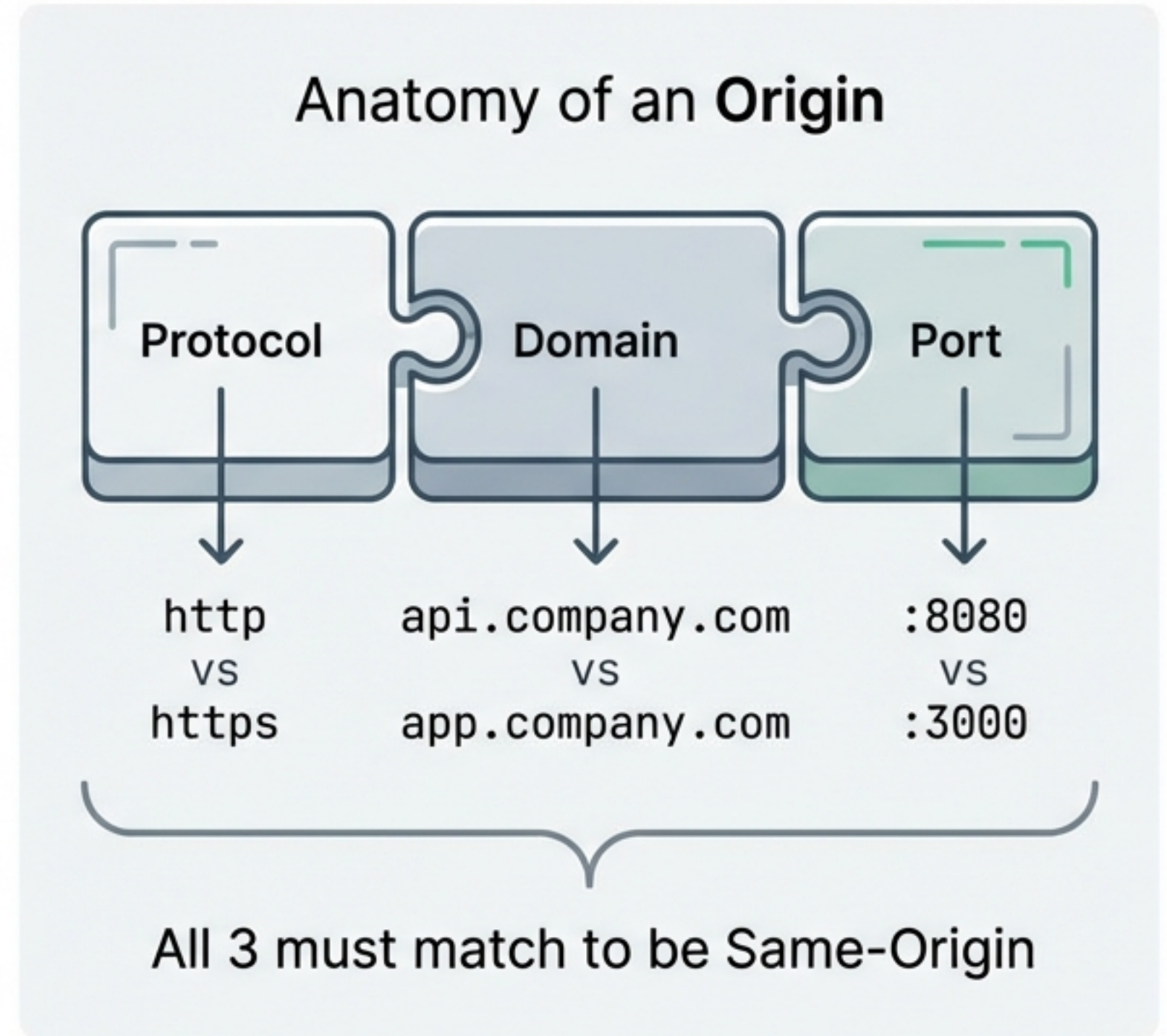
Bridging the gap between localhost:8080 and localhost:3000 securely and efficiently.

The Gatekeeper: Understanding the Same-Origin Policy

Cross-Origin Resource Sharing (CORS)

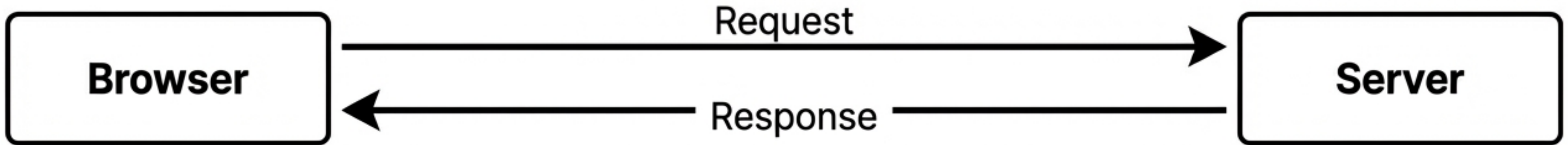
is a W3C standard that allows a browser to permit loading resources from an origin other than its own. The Browser is the enforcer, not the server.

If your Nuxt app is on port 3000 and Spring is on 8080, you are cross-origin. The browser will block the response unless the server explicitly permits it via headers.



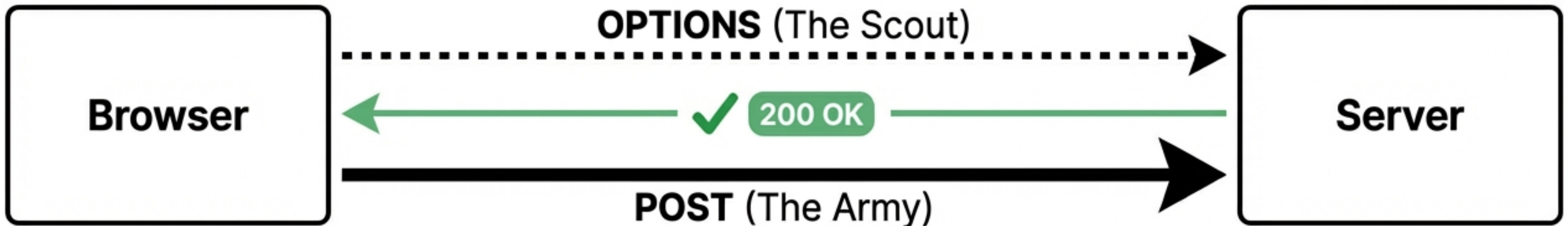
The Handshake: Simple vs. Preflight

Simple Requests (GET, POST)



Sent immediately. Browser checks "Access-Control-Allow-Origin" on return.

Preflight Requests (JSON, Custom Headers, PUT/DELETE)



Annotation: If the Scout (OPTIONS) fails (401/403), the Army never leaves camp.

Spring Boot Level 1: Fine-Grained Control

The Annotation Strategy

Best for: Rapid prototyping or enabling CORS on specific controllers.

Pros: Precise, easy to implement.

Cons: Hard to maintain, violates DRY (Don't Repeat Yourself), leads to scattershot configuration.

```
@RestController
@RequestMapping("/api/v1/users")
@CrossOrigin(origins = "http://localhost:3000") // <-- The Quick Fix
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }
}
```

The @CrossOrigin annotation applies rules only to this specific controller.

Spring Boot Level 2: Global MVC Configuration

The Centralized Strategy

Best for: Defining rules for the entire application in one place.

Implementation: Using the `WebMvcConfigurer` interface.

Key Insight: This creates a global rule set for all request mappings handled by the DispatcherServlet.

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**") // Applies to all endpoints
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```


Level 3: The 'Spring Security' Trap



The Problem: Standard CORS configurations operate at the DispatcherServlet level. Spring Security sits *before* that layer. If a Preflight (OPTIONS) request hits the Security Filter Chain without authentication (which browsers never send on preflights), **Spring Security rejects** it before it ever reaches your CORS config.

The Correct Fix: Security Filter Chain

For production apps using Spring Security, you must register CORS handling *inside* the Security chain. This allows the OPTIONS request to pass through the blockade.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    // The Hook: Registering the configuration source
    http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
    // ... other security configs
    return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:3000"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true); // Vital for cookies

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```


The 'Allow Credentials' Pitfall

You cannot use Wildcard `\*` origins if `allowCredentials` is set to `true`. This causes an `IllegalArgumentException` on startup.

✗ WRONG ✗

```
config.setAllowedOrigins(List.of("*"));
config.setAllowCredentials(true);
```

✗ **Browser Error:** Value of 'Access-Control-Allow-Origin' cannot be '*'.

✓ RIGHT ✓

```
// Option A: Specific Origin
config.setAllowedOrigins(List.of(
    "https://app.mydomain.com"));
config.setAllowCredentials(true);

// Option B: Dynamic Patterns (Spring Boot
// Boot 2.4+)
config.setAllowedOriginPatterns(List.
    of("https://*.mydomain.com"));
```

✓ Secure and Functional.

Operational Hygiene: Externalizing Config

Move hardcoded URLs out of compiled code to support multiple environments (Dev, Staging, Prod).

application.yml

```
app:
  cors:
    allowed-origins:
      "http://localhost:3000,"
      "https://staging.myapp.com"
```



Java Class

```
@Value("${app.cors.allowed-origins}")
private List<String> allowedOrigins;

// Usage
config.setAllowedOrigins(allowedOrigins);
```


The Frontend Dilemma: Nuxt 3



Origin:

`http://localhost:3000`

Action:

User initiates Login via Axios/Fetch.



The Conflict:

The browser sees Nuxt trying to talk to a stranger. Nuxt must be configured to send credentials (cookies) correctly.



Origin:

`http://localhost:8080`

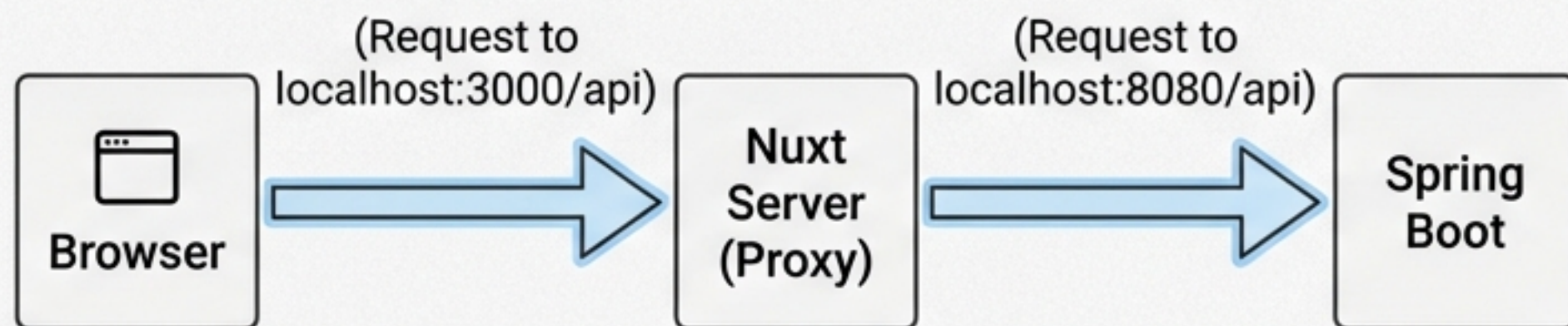
Action:

API Server receiving request.

Development Hack: The Nitro Proxy



Bypass the browser's CORS check entirely during development by tricking it. The browser talks to Nuxt (same origin), and Nuxt talks to Spring (server-to-server).



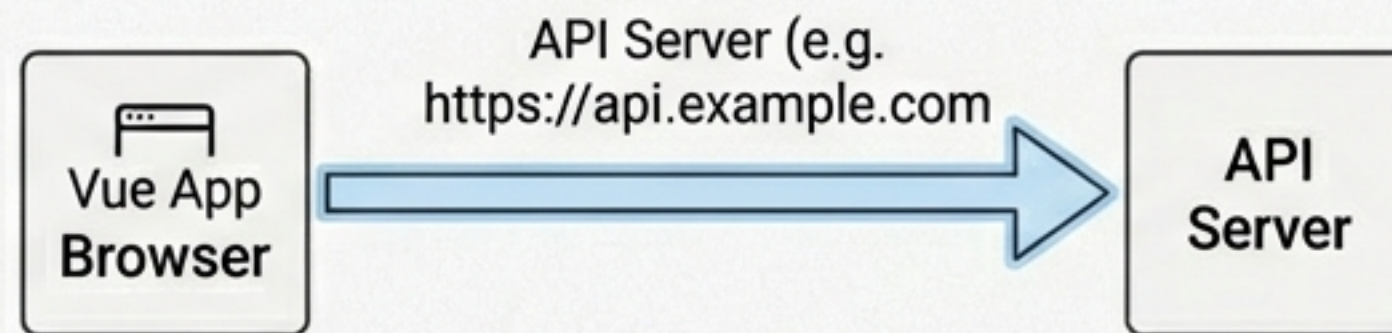
Server-to-Server communication
is NOT subject to CORS.

nuxt.config.ts

```
export default defineNuxtConfig({
  routeRules: {
    '/api/**': {
      // Proxies requests to backend
      proxy: 'http://localhost:8080/api/**'
    }
  }
})
```


Production Configuration: Runtime Config

In production, we don't proxy. We point directly to the API using environment variables.



nuxt.config.ts

```
runtimeConfig: {
  public: {
    apiBase: 'Nuxt_PUBLIC_API_BASE' // Overridden by .env
  }
}
```

Vue Component

```
const config = useRuntimeConfig()
const { data } = await useFetch(`
  ${config.public.apiBase}/users`, {
  credentials: 'include' // CRITICAL: Sends cookies/session ID
})
```



Note: Without `credentials: 'include'`, the browser will drop the session cookie even if CORS is valid.

Debugging the Handshake

Scenario 1: Preflight Blocked



```
>_ OPTIONS https://api.example.com/data 401 Unauthorized
```

Diagnosis:

Spring Security is blocking the preflight.

Fix:

Check SecurityFilterChain config (Slide 7).



Scenario 2: Headers Missing

```
>_ GET https://api.example.com/data 200 OK
Error: Missing Access-Control-Allow-Origin header
```

Diagnosis:

Backend processed request but didn't add headers.

Fix:

Check **allowedOrigins** list match (http vs https, trailing slashes).



Scenario 3: Wildcard with Credentials

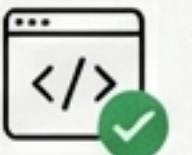
```
Error: The value of 'Access-Control-Allow-Origin' must not be '*' when requests are made with 'credentials: include'.
```

Diagnosis:

Wildcard used with credentials.

Fix:

Use `allowedOriginPatterns` instead (Slide 8).



The Cheat Sheet: Choosing Your Strategy

Scenario	Recommended Strategy	Implementation
Rapid Prototyping / Single Endpoint	Annotation	@CrossOrigin
Standard MVC App (No Security)	Global Config	WebMvcConfigurer

★ GOLD STANDARD

Production App with Spring Security	Security Bean	CorsConfigurationSource (in Security Chain)
Local Dev Friction	Proxy	Nuxt routeRules

Final Takeaways

1. **Safety First:** Never deploy `allowedOrigins('*')` to production if handling sensitive data.
2. **Consistency:** Match your `application.properties` and `.env` variables exactly (watch out for http/https mismatches).
3. **Distinction:** A 401 Unauthorized is an Auth error. A CORS error is a Browser Permission error. Don't confuse them.

Secure the handshake, and the data will flow.